

# **GUIDA ALL'IMPLEMENTAZIO NE DI UN SITO JEKYLL**

**LBD AI**

**Daniele Fadda  
Eleonora Cappuccio**

**Università di Pisa**  
Corso: Laboratorio di Big Data e AI  
Anno Accademico: 2025-2026

# Primi passi con il template Jekyll

## Cosa imparerete

- **Template Setup:** clonare e configurare il template Jekyll
- **Repository Management:** configurare repository GitHub e collaborator
- **Local Development:** ambiente di sviluppo locale
- **GitHub Pages:** pubblicare il sito automaticamente

## Prerequisiti

- Account GitHub

# Check dell'installazione

## Jekyll 3.9.3 o successive

`jekyll -v` per controllare la versione installata

## Ruby 3.1 o successive

`ruby -v` per controllare la versione di Ruby installata

# Passo 1: Configurazione del template di progetto

Questo passaggio deve essere eseguito da un componente solo del gruppo.

## Uso del repository template

1. **Vai su:** `https://github.com/danielefadda/master-projects-template-2026`
2. **Clicca:** pulsante "Use this template" (in alto a destra)
3. **Seleziona:** "Create a new repository"
4. **Formato nome:** `g0-2026-website` (sostituisci `g0` con il numero del tuo gruppo)
5. **Visibilit :** lascia Public
6. **Crea:** clicca "Create repository"

# Passo 2: Gestione dei collaboratori

## Aggiungere membri al team

1. **Impostazioni repository:** vai nelle impostazioni del repository
2. **Manage Access:** clicca "Manage access"
3. **Invite Collaborators:** aggiungi i membri tramite username GitHub o email
4. **Docenti richiesti:** aggiungi sempre: `danielefaddd` , `Elecapp` e `robertotrasarti`

# Passo 2: Gestione dei collaboratori

## Cosa faranno i collaboratori?

- **Clone repository:** scaricheranno il progetto in locale
- **Modifiche e commit:** apporteranno modifiche e le salveranno con commit
- **Push su GitHub:** invieranno le modifiche al repository remoto

## Buone pratiche di collaborazione

- **Lavorare su branch separati:** ogni task dovrebbe avere un branch dedicato
- **Commit frequenti:** salvate spesso le modifiche con messaggi chiari
- **Code review:** rivedete le modifiche prima del merge
- **Pull request:** usate PR per integrare le modifiche nel branch principale
- **Comunicazione chiara:** usa le GitHub Issues per tracciare i task

# Comprendere la struttura del progetto

## Struttura del repository

```
project-template/docs/  
├── _config.yml          # Impostazioni base del sito  
├── _data/  
│   └── settings.yml    # Configurazione visuale e contenuti  
├── _pages/  
├── assets/  
├── _includes/  
└── _layouts/           # Layout HTML
```

# Passo 3: Setup sviluppo locale (già fatto)

## Installazione Ruby e Jekyll

1. **Installa Ruby:** segui la [guida ufficiale Jekyll](#)
2. **Clona il repository:** `git clone [your-repo-url]`
3. **Installa le dipendenze:** `bundle install`
4. **Avvia il server:** `bundle exec jekyll serve`
5. **Accedi al sito:** `http://127.0.0.1:4000/[repo-name]`

## Vantaggi dello sviluppo locale

- **Test piu rapidi:** preview immediata delle modifiche
- **Lavoro offline:** non serve Internet durante lo sviluppo (eccetto per compilare il remote theme)
- **Editing avanzato:** usa il tuo IDE preferito



# Passo 4: Modifica file - Tre metodi

## Metodo 1: Editing diretto su GitHub

- **Modifiche semplici:** modifica file direttamente nel browser
- **Aggiornamenti rapidi:** ideale per contenuti testuali
- **Nessun setup:** no installazione locale

## Metodo 2: GitHub Codespaces

- **Editor avanzato:** IDE completo nel browser
- **Estensioni:** installa Prettier per formattazione codice

## Metodo 3: Sviluppo locale (consigliato)

- **Controllo completo:** usa qualsiasi editor (noi usiamo VS Code)
- **Integrazione Git:** version control avanzato
- **Lavoro offline:** *ni*, il tema è comunque in remoto, ma puoi fare modifiche senza connessione costante.

## Passo 5: Configurare \_config.yml

```
# Impostazioni repository
baseurl: "/g0-2026-website" # Nome del repository
url: "https://username.github.io" # URL GitHub Pages

# Informazioni sito
title: "Titolo del Progetto"
subtitle: "Sottotitolo del Progetto"
description: "Breve descrizione del progetto"

# Link repository (footer)
github_repo:
  - name: "Gruppo 0 - Repository del sito"
    url: "https://github.com/your-username/website-repo"
```

# Passo 6: Deployment con GitHub Pages

## Pubblicare il sito

1. **Crea il branch `gh-pages`**: vai su GitHub, clicca "Branch" e crea un nuovo branch: `gh-pages`
2. **Configura Pages**
  - vai in Settings > Pages.
  - In "Source", seleziona "Deploy from a branch" `gh-pages` e cartella `/ (root)`.
3. **Deploy tramite action**: nella tab "Actions", clicca sul workflow `Deploy to GitHub Pages` e poi "Run workflow" per avviare il deploy.

## URL del sito

- **Formato:** `https://username.github.io/repository-name/`
- **Esempio:** `https://johndoe.github.io/g0-2026-website/`

# Passo 6: Suggerimenti per il deployment GitHub Pages

## Suggerimenti per il deployment

- **Tempo di build:** considera *5-10 minuti* per il primo deploy
- **Aggiornamenti:** le successive modifiche compaiono in tempi più ragionevoli
- **Troubleshooting:** controlla la tab Actions per eventuali errori

# Passo 7: Creare e gestire le pagine

## Struttura pagine

```
docs/_pages/  
├── index.md          # Homepage  
├── project.md        # Descrizione progetto  
├── team.md           # Membri del team  
├── deliverables.md   # Risultati e download  
└── about.md          # Pagina custom (opzionale)
```

# Passo 7: Creare e gestire le pagine

## Creare una nuova pagina

1. **Crea file:** aggiungi un file `.md` nella directory `_pages/`
2. **Front Matter:** aggiungi header YAML con metadati
3. **Contenuto:** scrivi in formato Markdown
4. **URL:** il nome file diventa URL della pagina

# Passo 8: Configurazione Front Matter

## Front Matter base

```
---  
layout: default  
title: "Titolo Pagina"  
subtitle: "Sottotitolo Pagina"  
header_type: hero  
header_img: assets/images/header.png  
vega: true # Abilita i grafici  
---
```

# Passo 8: Configurazione Front Matter

## Tipi di header

- **hero**: header full-width con immagine di sfondo
- **base**: header standard
- **post**: header in stile blog
- **image**: header focalizzato sull'immagine
- **splash**: header full-screen ad alto impatto



# Passo 9: Creazione contenuti con Markdown

## Fondamenti Markdown

**# Titolo 1**

**## Titolo 2**

**\*\*Testo in grassetto\*\***

*\*Testo in corsivo\**

- Elenco puntato
- Un altro elemento

[Testo link](https://example.com)

## Styling avanzato

Questo è un paragrafo evidenziato.  
{: .lead}

Questo testo appare in verde.  
{: .green}

# Passo 10: Aggiungere immagini

## Gestione immagini

- **Posizione:** salva le immagini in `assets/images/`
- **Sintassi Markdown:** `![Testo alt](assets/images/image.jpg)`
- **Tag HTML:** ``

## Gallerie di immagini

```
<!-- Salva le immagini in assets/images/gallery_name/ -->
{% include_cached snippets/masonry.html internal="gallery_name" %}
```

## Configurazione gallery

```
# Nel file _config.yml
defaults:
  - scope:
      path: "assets/images/gallery_name"
    values:
      image_col: gallery_name
```

# Passo 11: Embedding dei video

## Integrazione YouTube

```
<!-- Shortcode video -->  
{% include video id="VIDEO_ID" provider="youtube" %}
```

## Provider video

- **YouTube**: scelta piu comune
- **Vimeo**: video professionali
- **Dailymotion**: piattaforma alternativa

# Passo 12: Aggiungere icone ed elementi visivi

## Icone Font Awesome

```
<i class="fa-solid fa-pen-nib"></i> Testo con icona  
<i class="fa-solid fa-code"></i> Icona codice  
<i class="fa-solid fa-chart-bar"></i> Icona grafico
```

## Risorse icone

- **Font Awesome:** [fontawesome.com](https://fontawesome.com)
- **Varieta:** migliaia di icone gratuite
- **Categorie:** tecniche, social, business, ecc.

## Suggerimenti d'uso

- Usa le icone per migliorare la leggibilità
- Mantieni coerenza nello stile delle icone
- Non abusarne: meno e meglio

## Passo 13: Creare modal interattivi

```
<!-- Pulsante trigger -->  
<button type="button" class="btn btn-primary"  
      data-toggle="modal" data-target="#myModal">  
  Apri Modal  
</button>
```

```
<!-- Struttura modal -->  
<div class="modal fade" id="myModal" tabindex="-1">  
  <div class="modal-dialog modal-xl modal-dialog-centered">  
    <div class="modal-content">  
      <div class="modal-header">  
        <h5 class="modal-title">Titolo Modal</h5>  
        <button type="button" class="btn-close"  
          data-bs-dismiss="modal"></button>  
      </div>  
      <div class="modal-body">  
        Contenuto del modal...  
      </div>  
    </div>  
  </div>  
</div>
```

# Passo 14: Aggiungere data visualization (1/3)

## Integrazione grafici (Vega-Altair)

1. **Crea il grafico:** genera JSON con Python/Altair
2. **Salva il file:** inseriscilo in `assets/charts/`
3. **Abilita Vega:** aggiungi `vega: true` nel front matter
4. **Embed grafico:** usa lo shortcode nella pagina

# Passo 14: Aggiungere data visualization (2/3)

## Implementazione del grafico

```
# Codice Python per generare il grafico
import altair as alt

chart = alt.Chart(data).mark_bar().encode(
    x='column_a:N',
    y='column_b:Q'
).properties(width='container', height=300)

chart.save('assets/charts/my_chart.json')
```



# Passo 14: Aggiungere data visualization (3/3)

## Embedding del grafico in Jekyll

```
<!-- Nella tua pagina -->  
<div style="height: 300px">  
  <vegachart schema-url="/repo-name/assets/charts/my_chart.json"  
    style="width: 100%; height: 100%"></vegachart>  
</div>
```

# Passo 15: Personalizzare la navigazione

```
# Nel file _config.yml
navbar:
  style: dual
  brand:
    title: "Titolo del Progetto"
    img: "./assets/images/logo.png"
    url: /
  nav:
    - title: Guida
      url: /guide.html
    - title: Progetto
      url: /project.html
    ...
```

```
...
- title: Team
  url: /team.html
- title: Risorse
  child:
    - title: Documentazione
      url: /docs.html
    - title: Download
      url: /downloads.html
```

# Passo 16: Personalizzazione del tema

## Scegliere un tema

```
# Nel file _config.yml
chulapa-skin:
  skin: "graymor" # oppure minty, lux, journal, ecc.
```

## Temi disponibili

- **graymor**: tema grigio professionale
- **minty**: tema verde fresco
- **lux**: tema scuro elegante
- ...

# Passo 16: Personalizzazione del tema

## Colori custom

```
chulapa-skin:  
  vars:  
    primary-color: "#ff0000"  
    body-bg: "#fbf1ed"
```

# Passo 17: Aggiungere font custom

## Integrazione Google Fonts

```
# Nel file _config.yml
googlefonts:
  - url: "https://fonts.googleapis.com/css2?family=Lekton:ital,wght@0,400;0,700;1,400&display=swap"

chulapa-skin:
  vars:
    headings-font-family: "Lekton"
```

# Passo 17: Aggiungere font custom in locale

## Classi CSS personalizzate

```
// In assets/css/custom.scss
.custom-font {
  font-family: 'Lekton', monospace;
  font-weight: 400;
}

.custom-font-bold {
  font-family: 'Lekton', monospace;
  font-weight: 700;
}
```

# Passo 18: Tecniche di layout avanzate

## Sezioni full-width

```
<div class="full-width-wrapper">  
    
</div>
```

# Passo 19: Gestione membri del team

## Configurazione team

```
# In _data/members.yml
- name: Nome Cognome
  github: https://kdd.isti.cnr.it/people/student-1
  role: student
```



# Passo 20: Checklist finale e best practice

## Checklist pre-lancio

-  **Nome repository:** formato corretto (gX-2026-website)
-  **Baseurl:** corrisponde al nome repository
-  **Front Matter:** tutte le pagine hanno header corretti
-  **Navigazione:** aggiornata e funzionante
-  **Info team:** informazioni membri complete
-  **Multimedia:** immagini e video posizionati correttamente
-  **GitHub Pages:** deployment completato con successo

# Passo 20: Checklist finale e best practice

## Garanzia di qualita

- **Cross-browser testing:** Chrome, Firefox, Safari
- **Mobile responsive:** test su dimensioni schermo diverse
- **Load speed:** ottimizza immagini e asset
- **Accessibility:** testo alt per immagini, heading corretti
- **SEO:** meta description, page title

# Grazie!

## Domande?

- Rivedi la documentazione della guida
- Controlla il repository GitHub